

[일러두기] 0 = false , 1 = true로 재정의하였습니다. 이 내용은 급하게 만들어진 내용이여서 정확하지 않은 내용이 일부 있을 수 있습니다

[기초 논리 연산]

기초 논리 연산에는 AND, OR, NOT이 있다.

AND연산 - AND연산은 2개 이상의 n개의 입력이 있을 시, 모든 입력이 1이면 출력 Y가 1로 출력되고, 이 외의 경우에는 출력 Y는 0으로 출력된다.

2개의 입력이 있는 AND연산의 진리표			3개의 입력이 있는 AND연산의 진리표			
입력		출력	입력			출력
A	B	Y	A	B	C	Y
0	0	0	0	0	0	0
0	1	0	0	0	1	0
1	0	0	0	1	0	0
1	1	1	0	1	1	0
			1	0	0	0
			1	0	1	0
			1	1	0	0
			1	1	1	1

OR연산 - OR연산은 2개 이상의 n개의 입력이 있을 시, 하나 이상의 입력이 1이면 출력 Y가 1로 출력되고, 이 외의 경우에는 출력 Y는 0으로 출력된다.

2개의 입력이 있는 OR연산의 진리표			3개의 입력이 있는 OR연산의 진리표			
입력		출력	입력			출력
A	B	Y	A	B	C	Y
0	0	0	0	0	0	0
0	1	1	0	0	1	1
1	0	1	0	1	0	1
1	1	1	0	1	1	1
			1	0	0	1
			1	0	1	1
			1	1	0	1
			1	1	1	1

NOT연산 - NOT연산은 1개의 입력을 반전시켜서 출력 Y로 내보내는 연산이다.

1개의 입력이 있는 NOT연산의 진리표	
입력	출력
A	Y
0	1
1	0

[응용 논리 연산]
기본적으로 XOR, NAND, NOR, XNOR을 알아두면 좋다.

XOR연산 - Exclusive OR라는 의미로 배타적 논리합을 구현하는 연산이다. 쉽게 말해 입력 2개에서 하나의 입력만 1일시 1을 출력하는 연산이다. 풀어서 말하자면 입력 2개의 값이 다를 때만 1이 출력되는 연산으로 이해하여도 좋다.

2개의 입력이 있는 XOR연산의 진리표		
입력		출력
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NAND연산 - NOT AND라는 의미로 AND연산을 수행한 후 NOT연산을 수행한 결과를 구현하는 연산이다. AND연산과 마찬가지로 NAND연산도 2개 이상의 n개의 입력을 넣을 수 있다. NAND연산은 모든 입력이 1일 때만 출력 Y에 0을 내보내는 연산이다.

2개의 입력이 있는 NAND연산의 진리표		
입력		출력
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

3개의 입력이 있는 NAND연산의 진리표			
입력			출력
A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

NOR연산 - NOT OR라는 의미로 OR연산을 수행한 후 NOT연산을 수행한 결과를 구현하는 연산이다. OR연산과 마찬가지로 NOR연산도 2개 이상의 n개의 입력을 넣을 수 있다. NOR연산은 모든 입력이 0일 때만 출력 Y에 1을 내보내는 연산이다.

2개의 입력이 있는 NOR연산의 진리표			3개의 입력이 있는 NOR연산의 진리표			
입력		출력	입력			출력
A	B	Y	A	B	C	Y
0	0	1	0	0	0	1
0	1	0	0	0	1	0
1	0	0	0	1	0	0
1	1	0	0	1	1	0
			1	0	0	0
			1	0	1	0
			1	1	0	0
			1	1	1	0

XNOR연산 - XOR연산을 수행한 후 NOT연산을 수행한 결과를 구현하는 연산이다. 그러므로 입력 2개가 같을 때만 출력 Y에 1을 출력하는 연산이다.

2개의 입력이 있는 XNOR연산의 진리표		
입력		출력
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

[불 대수에서의 수식]
여기에서는 덧셈연산자와 곱셈연산자를 재정의하여 사용할 것이다.
입력이 2개인 OR => +(덧셈연산자)
입력이 2개인 AND => ×(곱셈연산자)

여기에서 굳이 입력이 2개인 OR, AND라고 정의한 이유는 덧셈연산자와 곱셈연산자가 이항연산자이기 때문이다. 이항 연산자는 입력 2개에 출력인 하나인 함수라고 볼 수 있다. 이를 알기 쉽게 표현하자면
 $a+b \Leftrightarrow a+b$
 $a \times b \Leftrightarrow a \times b$
라고 할 수 있다. 그러므로 재정의할 때도 입력 2개에 출력 1개인 함수로 재정의하는 것이 일반적으로 적절하다.
그러므로 $a=1, b=0$ 일시 $a+b=1$ 이고 $ab=0$ 이다.
그리고 NOR연산은 $\oplus$ 라는 기호로 표현한다.

불 대수를 다룸에 있어서 기본적인 연산 표기법을 알아보자

연산	수식
입력이 2개인 AND	AB
입력이 2개인 OR	$A+B$
NOT	$\overline{A}$
XOR	$A\oplus B$
XNOR	$A\odot B$

불 대수를 다룸에 있어서 기본적인 연산 법칙을 알아보자

법칙	용도	AND	OR
교환법칙	수식	$AB=BA$	$A+B=B+A$
항등법칙	수식	$1A=A$	$0+A=A$
지배법칙	수식	$0A=0$	$1+A=1$
멱등법칙	수식	$AA=A$	$A+A=A$
역원법칙	수식	$A\overline{A}=0$	$A+\overline{A}=1$
결합법칙	수식	$(AB)C=A(BC)$	$(A+B)+C=A+(B+C)$
분배법칙	수식	$A+BC=(A+B)(A+C)$	$A(B+C)=AB+AC$
흡수법칙	수식	$A(A+B)=A$	$A+AB=A$
	증명	$A(A+B)=A\Leftrightarrow AA+AB=A$ $\Leftrightarrow A+AB=A\Leftrightarrow A(1+B)=A$ $\Leftrightarrow A=A$	$A+AB=A\Leftrightarrow (A+A)(A+B)=A$ $\Leftrightarrow A(A+B)=A\Leftrightarrow AA+AB=A$ $\Leftrightarrow A+AB=A\Leftrightarrow A(1+B)=A$ $\Leftrightarrow A=A$
드모르간 법칙	수식	$\overline{AB}=\overline{A}+\overline{B}$ $\overline{ABC}=\overline{A}+\overline{B}+\overline{C}$...	$\overline{A+B}=\overline{A}\overline{B}$ $\overline{A+B+C}=\overline{A}\overline{B}\overline{C}$...
법칙	용도	NOT	
항등 규칙	수식	$A=\overline{\overline{A}}=\overline{\overline{\overline{A}}}\dots$ $\Leftrightarrow A=NOT(NOT(A))=NOT(NOT(NOT(NOT(A))))\dots$ $\Leftrightarrow A=NOT^{2n}(A), n\in N^+$	

이 표에서 AND열에 적힌 내용과 OR열에 적힌 내용은 서로 드모르간 법칙에 따라 유도할 수 있다.

[기본적인 논리 연산 진리표를 이용하여 논리식 유도하기]
아까전에 $\oplus$ 로 정의한 XOR연산을 AND, OR연산만 이용해서 표현해보자.

2개의 입력이 있는 XOR연산의 진리표		
입력		출력
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

XOR의 진리표에서 Y=1인 경우는 XOR(0,1), XOR(1,0)때이다. 그리고 Y=1인 수식을 OR연산하면 다음과 같이 나온다.
 $XOR(A,B)=XOR(0,1)+XOR(1,0)$
여기에서 A, B에 대한 값 정의를 다음과 같이 하면

A의 값	수식	B의 값	수식
0	$\overline{A}$	0	$\overline{B}$
1	A	1	B

$$XOR(A,B) = \overline{A}B + A\overline{B}$$

로 표기할 수 있다. 여기에서 이 수식의 의미는 $XOR(0,1) \Rightarrow \overline{A}B$, $XOR(1,0) \Rightarrow A\overline{B}$ 라는 의미이다. 예시로 XOR(A,B)에서 XOR(0,1)을 입력하면, A=0, B=1이고 NOT(A)=1이 되기 때문에 $\overline{A}B$ 는 1이 되어서 이 함수의 결과 값은 1이 된다.

[논리식을 이용한 NAND, NOR연산의 Universal Gate확인]

Universal Gate는 모든 형태의 논리식을 표현할 수 있는 논리게이트이다. AND, OR, NOT논리 게이트를 조합하여서 이용하면 모든 논리식을 만들 수 있다. 다만 AND, OR, NOT게이트는 독립적인 한 종류의 게이트로는 모든 논리식을 만들 수 없으므로 Universal Gate라고 할 수 없다.

대표적인 Universal Gate에는 NAND, NOR게이트가 있다. 이를 확인해보자.

NAND, NOR게이트가 Universal Gate임을 확인하기 위해서는 각각의 게이트만으로 AND, OR, NOT논리 게이트를 표현할 수 있음을 보이면 된다.

NAND게이트의 Universal Gate 확인	
논리게이트	논리식/확인식
NAND	$\overline{AB}$
NOT	$\overline{AA} = \overline{A}$
AND	$\overline{\overline{ABAB}} = \overline{\overline{AB} + \overline{AB}} = \overline{\overline{AB}} = AB$
OR	$\overline{\overline{AABB}} = \overline{\overline{AB}} = \overline{\overline{A} + \overline{B}} = A + B$

NOR게이트의 Universal Gate 확인	
논리게이트	논리식/확인식
NOR	$\overline{A + B}$
NOT	$\overline{A + A} = \overline{A}$
AND	$\overline{\overline{A + A} + \overline{B + B}} = \overline{\overline{A} + \overline{B}} = \overline{\overline{A} \overline{B}} = AB$
OR	$\overline{\overline{A + B} + \overline{A + B}} = \overline{\overline{A + B}} = A + B$

그럼 이제 아래의 문제를 풀어보자.

AND, OR연산을 이용하여 표현한
수식 $F(A,B,C,D,E,F,G,H) = ABC + DE + F + G + H$ 를 NAND연산만의 표현으로 바꾸기

1. $\overline{\overline{ABCDEFHG}}$

2. $\overline{\overline{ABCDEFHG}}$

$F(A,B,C,D,E,F,G,H) = ABC + DE + F + G + H = \overline{\overline{ABCDEFHG}}$ 인지 확인해 보자

$\overline{\overline{ABCDEFHG}} = \overline{\overline{ABC} + \overline{DE} + \overline{F} + \overline{G} + \overline{H}} = ABC + DE + F + G + H$

$\therefore F(A,B,C,D,E,F,G,H) = ABC + DE + F + G + H = \overline{\overline{ABCDEFHG}}$

[논리 회로 만들기 - 1bit 반가산기]

지금부터는 위에서 배운 내용을 토대로 1bit 반가산기를 만들어 볼 것이다. 각 논리 회로에서 입력은 2개이며, Sum(자리 올림 없는 위치의 합)과 Carry(자리 올림)라는 출력 2개로 진행할 것이다. 여기에서 Sum과 Carry는 각각 S와 C로 표기할 것이다.

지금부터 입력 A, B에 따른 2진수 덧셈을 시행해보자.

2개 입력에 따른 1bit 2진수 덧셈 연산		
A	B	결과
0	0	00 ₍₂₎
0	1	01 ₍₂₎
1	0	01 ₍₂₎
1	1	10 ₍₂₎

입력이 2개인 1bit 덧셈은 $2^2 = 4$ 개의 경우의 수가 존재한다. 여기에서 최하 결과값은 $A=0, B=0$ 일 때 나오므로 $0_{(2)}$ 이고, 최고 결과값은 $A=1, B=1$ 일 때 나오므로 $10_{(2)}$ 인 것을 알 수 있다.

LSB(최하위 비트)를 0번째로 두고 MSB(최상위 비트)로 다가갈수록 숫자가 커지는 방식으로 위치를 정하면, S는 LSB인 0번째 비트이고 C는 MSB인 1번째 비트임을 알 수 있다. 이를 바탕으로 진리표를 작성하면 다음과 같다.

2개 입력에 따른 1bit 2진수 반가산기 진리표			
입력		출력	
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

이를 바탕으로 S와 C의 논리식을 구하면 다음과 같다.

$$S = \overline{A}B + A\overline{B} = A \oplus B$$

$$C = AB$$

보시다시피 S는 XOR연산을, C는 AND연산으로 구할 수 있음을 알 수 있다.

반가산기를 통해 n_1, n_2 의 두 개의 2진수의 최하위 비트(LSB)의 합을 구할 수 있다.

[논리회로 만들기 - 1bit 전가산기]

이제 반가산기에서 배운 내용을 토대로 전가산기를 만들어보자. 여기에서는 CL(L은 Last)라는 지금 n번째 비트에서 덧셈을 수행하기 전에 n-1번째 비트에서 덧셈을 수행하고 올라온 자리 올림이 입력으로 추가되었다. 그러므로 혼란을 방지하기 위해 반가산기에서 C라고 표현했던 출력은 CF(F는 Future)로 표현하겠다.
이번에도 마찬가지로 3개의 입력 A, B, CL에 따른 2진수 덧셈을 시행해보자.

3개 입력에 따른 1bit 2진수 덧셈 결과			
A	B	CL	결과
0	0	0	00 ₍₂₎
0	0	1	01 ₍₂₎
0	1	0	01 ₍₂₎
0	1	1	10 ₍₂₎
1	0	0	01 ₍₂₎
1	0	1	10 ₍₂₎
1	1	0	10 ₍₂₎
1	1	1	11 ₍₂₎

입력이 2개인 1bit 덧셈은 $2^3 = 8$ 개의 경우의 수가 존재한다. 여기에서 최하 결과값은 A=0, B=0, CL=0일 때 나오므로 0₍₂₎이고, 최고 결과값은 A=1, B=1, CL=1일 때 나오므로 11₍₂₎인 것을 알 수 있다.
그럼 마찬가지로 진리표를 작성해보자.

3개 입력에 따른 1bit 2진수 전가산기 진리표				
입력			출력	
A	B	CL	CF	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

이를 바탕으로 S와 CF의 논리식을 구하면 다음과 같다.

$$S = \overline{A}\overline{B}C_L + \overline{A}B\overline{C}_L + A\overline{B}\overline{C}_L + ABC_L$$

$$C_F = \overline{A}BC_L + \overline{A}\overline{B}C_L + A\overline{B}C_L + ABC_L$$

이제 이 수식을 간략화해보자

먼저 S를 간략화 해보자

$$S = \overline{A}\overline{B}C_L + \overline{A}B\overline{C}_L + A\overline{B}\overline{C}_L + ABC_L \text{에서}$$

1. $A, \overline{A}$ 기준으로 묶는다

$$A(\overline{B}\overline{C}_L + B\overline{C}_L) + \overline{A}(\overline{B}\overline{C}_L + B\overline{C}_L)$$

2. 각각 XOR, XNOR을 적용한다

$$A(B \odot C_L) + \overline{A}(B \oplus C_L) = A(\overline{B \oplus C_L}) + \overline{A}(B \oplus C_L)$$

3. XOR형태이므로 다시 정리한다

$$A \oplus (B \oplus C_L)$$

4. XOR은 결합법칙이 성립하므로 괄호를 푼다

$$A \oplus B \oplus C_L$$

$$\therefore S = A \oplus B \oplus C_L$$

이제 CF를 간략화 해보자

$$C_F = \overline{A}BC_L + A\overline{B}C_L + A\overline{B}\overline{C}_L + ABC_L \text{에서}$$

1. $\overline{A}BC_L + A\overline{B}C_L$ 에 대해서 A로 묶은 후, $\overline{B}C_L, B\overline{C}_L$ 부분을 XOR로 처리한다

$$\overline{A}BC_L + A(B \oplus C_L) + A\overline{B}\overline{C}_L$$

2. BC_L 기준으로 묶는다

$$BC_L(\overline{A} + A) + A(B \oplus C_L)$$

3. 식을 정리한다

$$\therefore C_F = A(B \oplus C_L) + BC_L$$

참고로 $\overline{A}BC_L + A\overline{B}C_L$ 를 C_L 로 묶고 XOR한 후 풀으면 다음과 같은 결과가 나오고 그 결과도 맞다

$$\therefore C_F = C_L(A \oplus B) + AB$$

다른 방법을 알아보자

1. 한 입력에 대해서 0,1이 모두 존재하는 값을 묶고 정리한다

$$(A + \overline{A})BC_L + A\overline{B}C_L + A\overline{B}\overline{C}_L = BC_L + A\overline{B}C_L + A\overline{B}\overline{C}_L$$

2. C_L 로 묶고 정리한다

$$C_L(B + A\overline{B}) + A\overline{B}\overline{C}_L = C_L(B + A)(B + \overline{B}) + A\overline{B}\overline{C}_L = C_L(B + A) + A\overline{B}\overline{C}_L$$

3. 다시 풀고 A에 대해서 정리한다

$$AC_L + BC_L + A\overline{B}\overline{C}_L = A(C_L + \overline{B}\overline{C}_L) + BC_L = A(C_L + B)(C_L + \overline{C}_L) + BC_L = A(C_L + B) + BC_L$$

$$\therefore AC_L + AB + BC_L$$

CF는 다양한 방법으로 풀 수 있으며, 그 결과도 다양하게 나올 수 있음을 알 수 있다. 이럴 때는 회로를 구현하기 쉬운 논리식으로 회로를 구성하면 된다.

전가산기를 통해 n_1, n_2 의 두 개의 2진수의 최하위 비트(LSB)를 포함한 위치에 있는 각 비트의 합을 구할 수 있다.

[자연수 2진수 읽기]
LSB(최하위 비트)를 0번째로 두고 MSB(최상위 비트)로 다가갈수록 숫자가 커지는 방식으로 위치를 정해보자. 그럴시 다음 2진수는 다음과 같다.

1011011101 ₍₂₎										
위치	MSB(9)	8	7	6	5	4	3	2	1	LSB(0)
해당 위치의 숫자	1	0	1	1	0	1	1	1	0	1

여기에서 1011011101₍₂₎를 10진수로 변환해보자

2진수	1011011101 ₍₂₎									
위치(n)	MSB(9)	8	7	6	5	4	3	2	1	LSB(0)
해당 위치의 숫자	1	0	1	1	0	1	1	1	0	1
2 ⁿ	512	256	128	64	32	16	8	4	2	1
2 ⁿ ×해당 위치의 숫자	512	0	128	64	0	16	8	4	0	1
10진수	512+128+64+16+8+4+1=733									

여기에서 알면 좋은 상식으로 자연수 2진수를 MSB방향으로 한 칸 옮기면 값이 2배 커지고, LSB방향으로 한 칸 옮기면 값이 $\left\lfloor \frac{2진수 값}{2} \right\rfloor$ 가 된다.

[필요한 논리식 만들어보기]
지금부터는 함수 F(A,B,C,...,Z)에 대하여 입력값 A,B,C,...,Z을 맨 마지막 입력값(여기에서는 Z)을 LSB, 맨 처음 입력값(여기에서는 A)을 MSB로 하는 2진수로 표현한다. 그리고 함수F(A,B,C,...,Z)의 결과값이 1이 나오는 ‘입력값을 기반으로 하여 만들어진 2진수’들을 집합 T에 포함한다. 또한 F(A,B,C,...,Z)에서 1이 나오는 입력값들의 집합과 함수 F와의 관계는 다음과 같이 표현한다.

T_n은 집합 T에서 n번째에 있는 숫자
t_{n,m}은 T_n의 2진수에서 m번째 입력값(예시 : T₂ = 11010110⇒t_{2,3} = 0, t_{2,0} = 0)

$$F(A,B,C,...,Z) = \Sigma(T_1, T_2, T_3,..., T_n) = \Sigma(T)$$

여기에서 집합 T에 속한 각 2진수 T_n에 대해서 각 입력값 A_n,B_n,C_n,...,Z_n이 0일시 $\overline{t_{n,m}}$, 1일시 t_{n,m}로 표기하고 각 입력값 A_n,B_n,C_n,...,Z_n⇔t_{n,m},t_{n,m-1},t_{n,m-2},...,t_{n,0}을 곱하여 T_n을 구현하고, 각 T_n들에 대하여 더하면 등식이 성립된다.

만약 F(A,B,C,D)=Σ(0,5,10)일시 F(0,0,0,0), F(0,1,0,1), F(1,0,1,0)일 때 결과값이 1이 나온다는 의미이다.

그런가 하면 해당 숫자일 때 반드시 결과값이 1이어야 하는 값(주항)도 있지만, 해당 숫자에서 결과값이 0이거나 1이어도 상관없는 무관항도 있다. 주항 집합을 T, 무관항 집합을 D라고 했을 때 다음과 같이 표시한다.

$$F(A,B,C,...,Z) = \Sigma m(T_1, T_2, T_3,..., T_n) + \Sigma d(D_1, D_2, D_3,..., D_n) = \Sigma m(T) + \Sigma d(D)$$

이때 무관항이 없는 함수일 경우 주항 표시를 $F(A,B,C,...,Z) = \Sigma(T_1, T_2, T_3,..., T_n) = \Sigma(T)$ 와 같이 Σm이 아닌 Σ으로 축약해서 쓸 수 있다.

만약 위의 표시를 등식으로 바꾸기 위해서는 위에서 설명했던 집합 T를 변환하는 방법처럼 주항 집합을 변환하고, 무관항은 집합 D의 요소의 일부 또는 전체를 주항 집합을 변환하듯이 변환하거나 아예 없는 존재 즉 0으로 처리하고, 주항 집합 변환부와 무관항 집합 변환부를 더하면 등식이 성립한다.

그럼 지금부터 필요한 논리식을 만들어보겠다. $F(A,B,C,D)=\sum m(0,1,3,6,12)$ 인 함수가 필요하다고 가정하자. 이를 논리 회로로 표현하기 위해서는 편의를 위해 최대한 간략화된 논리식을 구하여야 한다. 이를 구하는 일반적인 방법에 대해서 알아보자.

논리식을 구하고 간략화하는 방법1

- 1. 진리표를 작성한다.
- 2. 진리표에서 결과값이 1인 집합 T를 구한다.
- 3. T_n 의 구성요소를 곱하고, 곱한 T의 요소들을 더한다.(집합 T를 논리식으로 변환)
- 4. 불 대수의 법칙들을 이용해 논리식을 간략화한다.

위의 내용을 따라해보겠다.

1. 진리표 작성

F(A,B,C,D)= $\sum(0,1,3,6,12)$ 의 진리표					
10진수로 변환한 입력 2진수	입력				출력
	A	B	C	D	
0	0	0	0	0	1
1	0	0	0	1	1
2	0	0	1	0	0
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	1
7	0	1	1	1	0
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	0
12	1	1	0	0	1
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	0

2. 집합 T 구하기

$T=\{0000_{(2)},0001_{(2)},0011_{(2)},0110_{(2)},1100_{(2)}\}$

3. 집합 T를 논리식으로 변환하기

$$F(A,B,C,D) = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD + A\overline{B}\overline{C}\overline{D}$$

4. 논리식 간략화

$$\begin{aligned} F(A,B,C,D) &= \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD + A\overline{B}\overline{C}\overline{D} \\ &= \overline{A}\overline{B}\overline{C}(\overline{D} + D) + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD = \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD + A\overline{B}\overline{C}\overline{D} \\ &= \overline{A}\overline{B}(\overline{C} + CD) + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}CD = \overline{A}\overline{B}(\overline{C} + C)(\overline{C} + D) + \overline{A}\overline{B}\overline{C}D + A\overline{B}\overline{C}\overline{D} \\ &= \overline{A}\overline{B}(\overline{C} + D) + \overline{A}\overline{B}\overline{C}D + A\overline{B}\overline{C}\overline{D} = \overline{A}\overline{B}(\overline{C} + D) + \overline{B}D(\overline{A}C + A\overline{C}) \\ &= \overline{A}\overline{B}(\overline{C} + D) + \overline{B}D(A \oplus C) \\ \therefore F(A,B,C,D) &= \overline{A}\overline{B}(\overline{C} + D) + \overline{B}D(A \oplus C) \end{aligned}$$

논리식을 구하고 간략화하는 방법2

1. 진리표를 작성한다.
2. 진리표에서 결과값이 1인 집합 T를 구한다.
3. 카르노맵을 그리고 집합 T에 해당하는 숫자에는 1을 아닌 집합에는 0을 적는다.
4. 카르노맵의 규칙에 따라 간략화한다.

지금은 카르노맵을 배우는 단계가 아니므로 카르노맵이 아닌 그 기본 원리를 이용하여서 논리식을 구해보겠다.
카르노맵의 기본 원리는 다음과 같다.

변수 G , 진함수 $F(A,B,C,...,G,...,Z)$,
 F 에서 G 를 제외한 함수 $f(A,B,C,...,\overline{G},...Z)$, 가 있을 시

$$\begin{aligned} T_{F_{n0}} &= com(T_{f_n}, \overline{G}, G \text{의 bit 번호}) = A_{f_n}, B_{f_n}, C_{f_n}, ..., \overline{G}, ..., Z_{f_n} \\ T_{F_{n1}} &= com(T_{f_n}, G, G \text{의 bit 번호}) = A_{f_n}, B_{f_n}, C_{f_n}, ..., G, ..., Z_{f_n} \end{aligned}$$

$$\begin{aligned} F(T_{F_{n0}}) &= F(T_{F_{n1}}) \Rightarrow f(T_{f_n}) = F(T_{F_{n0}}) = F(T_{F_{n1}}) \\ F(T_{F_{n0}}) &\neq F(T_{F_{n1}}) \Rightarrow f(T_{f_n}) = \emptyset \end{aligned}$$

$$\begin{aligned} f(T_{f_n}) &\neq \emptyset \wedge f(T_{f_n})G + f(T_{f_n})\overline{G} = f(T_{f_n})(G + \overline{G}) = f(T_{f_n}) \\ \Rightarrow F(T_{F_{n0}}) &= f(T_{f_n}) \wedge F(T_{F_{n1}}) = f(T_{f_n}) \Leftrightarrow F(T_{F_{n0}}) = F(T_{F_{n1}}) \end{aligned}$$

$$\therefore f(T_{f_n}) \neq \emptyset \Leftrightarrow F(T_{F_{n0}}) = F(T_{F_{n1}})$$

그러므로 $T = \{0000_{(2)}, 0001_{(2)}, 0011_{(2)}, 0110_{(2)}, 1100_{(2)}\}$ 에서 서로 하나씩만 겹치는 것을 뽑으면 $\{0000, 0001\}$, $\{0001, 0011\}$ 이 있다. 그래서 $000 \times_{(2)}, 00 \times 1_{(2)}, 0110_{(2)}, 1100_{(2)}$ 로 두고 변환하면 다음과 같이 된다.

$\overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}D + \overline{A}B\overline{C}\overline{D} + A\overline{B}\overline{C}\overline{D}$ 여기에서 간략화 하면

$$= \overline{A}\overline{B}(\overline{C} + D) + \overline{B}D(\overline{A}C + A\overline{C}) = \overline{A}\overline{B}(\overline{C} + D) + \overline{B}D(A \oplus C)$$

$$\therefore F(A,B,C,D) = \overline{A}\overline{B}(\overline{C} + D) + \overline{B}D(A \oplus C)$$

[다시 보는 논리 연산]

지금부터 OR, IMPLY 논리 연산을 진리표를 작성하여 논리식을 구해볼 것이다.

IMPLY 연산은 우리가 흔히 알고 있는 조건문 $A \rightarrow B$ 에 대한 참과 거짓을 내놓는 연산이다. 그럼 OR, IMPLY 연산의 진리표를 작성해보자.

2개의 입력이 있는 OR연산의 진리표			IMPLY연산의 진리표		
입력		출력	입력		출력
A	B	Y	A	B	Y
0	0	0	0	0	1
0	1	1	0	1	1
1	0	1	1	0	0
1	1	1	1	1	1

여기에서 IMPLY연산은 $(A \rightarrow B) \equiv (\neg A \vee B)$ 이다. 즉 지금까지 배웠던 것으로 표현하자면 $\overline{A} + B$ 이다. 그렇다면 위에서 배운 논리식을 간략화하는 방법으로 OR, IMPLY연산의 결과가 동일하게 나오는지 확인해보자.

OR연산 논리식 유도 및 간략화

$\overline{A}B + A\overline{B} + AB$ 에서

$$A(B + \overline{B}) + \overline{A}B = A + \overline{A}B = (A + \overline{A})(A + B) = A + B$$

$$\therefore A + B$$

그리고 이 유도 과정에서 $A \oplus B + AB = A + B$ 임을 알 수 있다

위의 유도 과정을 통해 처음 배웠던 OR연산 표기법과 같은 결과가 나옴을 알 수 있다. 그렇다면 3개의 입력을 가진 OR에 대해서도 확인해보자.

3개의 입력이 있는 OR연산의 진리표			
입력			출력
A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

위의 진리표에서 $\overline{A}\overline{B}C + \overline{A}B\overline{C} + \overline{A}BC + A\overline{B}\overline{C} + A\overline{B}C + AB\overline{C} + ABC$ 에서

$$\overline{A}(\overline{B}C + B\overline{C} + BC) + A(\overline{B}\overline{C} + \overline{B}C + B\overline{C} + BC)$$

여기에서 $f(s)G + f(s)\overline{G} = f(s) \neq \phi \Leftrightarrow F(T_{\overline{G}}) = F(T_G)$ 와 입력이 2개인 OR연산의 정의에 의해

$\overline{A}(B + C) + A$ 에서 이를 정리하면

$$B + C \equiv G$$

$$\overline{A}G + A = (\overline{A} + A)(G + A) = G + A = B + C + A$$

$$\therefore A + B + C$$

[참고]

$F =$ 전함수, $S = F$ 의 정의역, $\{0,1\} = F$ 의 치역, $f_{\text{변수}} = F$ 에서 변수가 빠진 함수에서 $f_{\text{변수}}$ 는 F 과 똑같이 정의역으로 S 을 가지며 다음과 같이 정의된다.

$com(S_n, \text{변수}), coms(S_n, \text{변수 조작 집합})$
 S 의 n 번째 요소인 숫자를 특정 변수부분만 1로 조작하는 함수
 (3번째 입력이 변수일시 0으로 조작함)
 예시)
 $F(A, B, C, D)$ 에서 $S_5 = 0101_{(2)}$ 일시
 $F(com(S_5, C)) = F(S_7) = F(0111_{(2)})$
 $F(com(S_5, B)) = F(S_5) = F(0101_{(2)})$
 $F(com(S_5, \overline{B})) = F(S_1) = F(0001_{(2)})$

$f_{\text{변수}}(S_n) = \begin{cases} F(com(S_n, \text{변수})) = F(com(S_n, \overline{\text{변수}})), & F(S_n) \\ F(com(S_n, \text{변수})) \neq F(com(S_n, \overline{\text{변수}})), & \phi \end{cases}$
 여기에서 ϕ 는 값을 특정할 수 없다는 의미이다.

이것을 이용해 다음을 알 수 있다.
 $S_n = s$ 일시
 $F(com(s, G)) = F(com(s, \overline{G})) \Leftrightarrow f_G(s) \neq \phi \wedge (f_G(s) = 0 \vee f_G(s) = 1)$
 $f_G(s) = \phi \Leftrightarrow F(com(s, G)) \neq F(com(s, \overline{G}))$
 $f_G(s) \neq \phi \Leftrightarrow f_G(com(s, G)) = f_G(com(s, \overline{G}))$
 $f_G(s)G + f_G(s)\overline{G} = f_G(s) \neq \phi \Leftrightarrow F(com(s, G)) = F(com(s, \overline{G}))$

여기에서 $f_{G,H} = f_G$ 에서 변수 H 가 빠진 함수이고 정의는 다음을 따른다고 해보자
 $f_{G,H}$ 의 정의는 다음과 같다.

$$f_{G,H}(S_n) = \begin{cases} f_G(com(s, H)) = f_G(com(s, \overline{H})) \wedge f_G(s) \neq \phi, & f_G(s) \\ f_G(s) = \phi, & \phi \\ f_G(com(s, H)) \neq f_G(com(s, \overline{H})), & \phi \end{cases}$$

$$\begin{aligned} f_{G,H}(s) \neq \phi &\Rightarrow f_G(s) \neq \phi \\ f_{G,H}(s) \neq \phi &\Leftrightarrow f_{G,H}(s)H + f_{G,H}(s)\overline{H} = f_G(s) \neq \phi \\ f_{G,H}(s) \neq \phi &\Leftrightarrow f_{G,H}(s)\overline{GH} + f_{G,H}(s)\overline{G}\overline{H} + f_{G,H}(s)G\overline{H} + f_{G,H}(s)GH = F(s) \neq \phi \end{aligned}$$

지금부터는 IMPLY연산의 논리식을 유도하고 간략화해보겠다.

IMPLY연산의 진리표		
입력		출력
A	B	Y
0	0	1
0	1	1
1	0	0
1	1	1

위의 진리표에서 $\overline{A}\overline{B} + \overline{A}B + AB$ 에서 정리하면

$$\overline{A}(\overline{B} + B) + AB = \overline{A} + AB = (\overline{A} + A)(\overline{A} + B)$$

$$\therefore \overline{A} + B$$

위의 유도 과정을 통해 $\neg A \vee B$ 와 같은 결과가 나옴을 알 수 있다.

[논리식 간략화 조금 더 나가보기]

지금까지 논리식을 적는 방법은 SOP(Sum of Products)으로써 변수를 모두 곱한 후 더하는 식으로 표기하는 방법을 사용하였다. 이 방식은 결과가 1인 결과값들로 해당 함수의 논리식을 적을 수 있는 방법이었다. 그렇다면 결과가 0인 결과값들로 논리식을 만드는 방법이 있을까? 결과부터 말하자면 있다. POS(Product of Sums)으로써 변수들을 모두 더하고 난 후 곱하는 방법이다. 이 방법을 이용해 결과가 0인 값들을 사용해 논리식을 만들 수 있다. POS방식은 함수에서 결과가 1인 경우보다 0이 더 적을 때 유용하다. 그렇다면 POS를 어떻게 쓰는지 배워보자.

$F(A,B,C,D)=\Sigma(0,1,2,3,5,6,8,11,12,15)$ 라는 함수가 있다고 해보자. 이 함수는 1인 결과가 10개나 있는 함수로써 지금까지 써왔던 SOP방식을 쓰기에는 너무 길어진다. 그러므로 POS방식을 쓰기 위해서 하나 꼽수를 써보려고 한다.

F(A,B,C,D)= $\Sigma(0,1,2,3,5,6,8,11,12,15)$ 의 진리표					
10진수로 변환한 입력 2진수	입력				출력
	A	B	C	D	
0	0	0	0	0	1
1	0	0	0	1	1
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	0
8	1	0	0	0	1
9	1	0	0	1	0

10	1	0	1	0	0
11	1	0	1	1	1
12	1	1	0	0	1
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	1

위에 있는 진리표는 F(A,B,C,D)=Σ(0,1,2,3,5,6,8,11,12,15)의 진리표이다. 이 함수 F 결과값에 NOT연산을 취해보자. 그러면 아래의 진리표와 같이 바뀐다.

F'(A,B,C,D)=Σ(4,7,9,10,13,14)의 진리표					
10진수로 변환한 입력 2진수	입력				출력
	A	B	C	D	
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	0
3	0	0	1	1	0
4	0	1	0	0	1
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	1
10	1	0	1	0	1
11	1	0	1	1	0
12	1	1	0	0	0
13	1	1	0	1	1
14	1	1	1	0	1
15	1	1	1	1	0

F는 F'가 되면서 F'(A,B,C,D)=Σ(4,7,9,10,13,14)로 바뀌었다. 그렇다면 F'의 SOP를 구하고 NOT연산을 취하면 결과적으로 F와 같은 결과가 나올 것이다. 한번 확인해보자.

$$\overline{F}(A,B,C,D) = \Sigma(4,7,9,10,13,14), F = \overline{\overline{F}}$$

$$\begin{aligned} \overline{F} &= \overline{AB\overline{C}\overline{D}} + \overline{AB\overline{C}D} + \overline{A\overline{B}C\overline{D}} + \overline{A\overline{B}CD} + \overline{AB\overline{C}\overline{D}} + \overline{AB\overline{C}D} \\ F &= (\overline{AB\overline{C}\overline{D}} + \overline{AB\overline{C}D} + \overline{A\overline{B}C\overline{D}} + \overline{A\overline{B}CD} + \overline{AB\overline{C}\overline{D}} + \overline{AB\overline{C}D}) \\ &= (\overline{\overline{A} + \overline{B} + \overline{C} + \overline{D}})(\overline{\overline{A} + \overline{B} + \overline{C} + \overline{D}})(\overline{\overline{A} + \overline{B} + \overline{C} + \overline{D}})(\overline{\overline{A} + \overline{B} + \overline{C} + \overline{D}})(\overline{\overline{A} + \overline{B} + \overline{C} + \overline{D}})(\overline{\overline{A} + \overline{B} + \overline{C} + \overline{D}}) \\ &= (A + \overline{B} + C + D)(A + \overline{B} + \overline{C} + \overline{D})(\overline{A} + B + C + \overline{D})(\overline{A} + B + \overline{C} + D)(\overline{A} + \overline{B} + C + \overline{D})(\overline{A} + \overline{B} + \overline{C} + D) \end{aligned}$$

위 논리식에서 최종적으로 나온 결과가 POS방식으로 논리식을 구하는 방법이며 SOP, POS방식일시 변수를 적는 방법은 다음과 같다.

변수(A)의 값	SOP수식 일시	POS수식 일시
0	$\overline{A}$	A
1	A	$\overline{A}$

이때 POS수식에서도 SOP수식과 비슷하게 다음이 적용된다.

$$(f + G)(f + \overline{G}) = ff + fG + f\overline{G} + G\overline{G} = f + fG + f\overline{G} = f + f(G + \overline{G}) = f + f = f$$
$$(f_G(s) + G)(f_G(s) + \overline{G}) = f_G(s) \neq \phi \Leftrightarrow F(\text{com}(s, G)) = F(\text{com}(s, \overline{G}))$$

위의 규칙들을 입력이 2개, 3개인 AND연산을 POS수식으로 적어서 확인해보자

$$AB = (A + B)(A + \overline{B})(\overline{A} + B)$$

1번 풀이법 – 분배법칙 이용

$$(A + B)(A + \overline{B})(\overline{A} + B) = (A + B)(A\overline{A} + B\overline{B} + AB + \overline{A}\overline{B}) = (A + B)(AB + \overline{A}\overline{B})$$
$$= AAB + A\overline{A}\overline{B} + ABB + \overline{A}\overline{B}B = AB + AB$$
$$\therefore AB$$

2번 풀이법 – $(f + G)(f + \overline{G}) = f$

$$(A + B)(A + \overline{B})(\overline{A} + B) = A(\overline{A} + B) = A\overline{A} + AB$$
$$\therefore AB$$
$$ABC = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C)(A + \overline{B} + \overline{C})(\overline{A} + B + C)(\overline{A} + B + \overline{C})(\overline{A} + \overline{B} + C)$$

에서 $(f + G)(f + \overline{G}) = f$ 를 적용하여 정리하면

$$(A + B)(A + \overline{B})(\overline{A} + B)(\overline{A} + \overline{B} + C) = A(\overline{A} + B)(\overline{A} + \overline{B} + C) = (A\overline{A} + AB)(\overline{A} + \overline{B} + C)$$
$$= AB(\overline{A} + \overline{B} + C) = A\overline{A}B + ABB + ABC$$
$$\therefore ABC$$

[논리식을 $F(A,B,C,\dots,D) = \sum m(n_1,n_2,n_3,\dots,n_m)$ 꼴로 바꾸기]

지금까지는 함수 F에 대한 논리식을 간결하게 만드는 방법들에 대해서 배웠다. 이번에는 간략하게 간결한 논리식을 F에서 1이 나오는 집합 T를 구하는 방법에 대해서 배울 것이다.

$F(A,B,C,D) = A(B + C) + A \oplus B$ 라는 논리식을 보자. 일단 논리식에 D가 없으므로 이 함수는 D의 입력값에 영향을 받지 않는 함수라는 것을 알 수 있다. 이제 이 논리식을 SOP형식으로 정리해보자. SOP형식으로 정리하면 아래와 같아진다.

$$F(A,B,C,D) = AB + AC + \overline{A}B + \overline{A}\overline{B}$$

여기에서 SOP형식을 지키면서 간략화를 해보자. 그럼 아래의 결과가 나온다.

$$F(A,B,C,D) = A + \overline{A}\overline{B}$$

여기에서 A항은 B,C,D의 입력값과 무관하고 A의 입력값으로만 결정되고, A`B항은 C,D의 입력값과 무관하게 A,B의 입력값에만 영향을 받음을 알 수 있다.

이제 이 SOP형식의 논리식을 2진수로 바꿔보자

$$A \Rightarrow 1 \times \times \times_{(2)}$$

$$\overline{A}B \Rightarrow 01 \times \times_{(2)}$$

여기에서 x라는 부분은 0이나 1이여도 무관하다는 것이므로, 한 2진수에서 x의 개수에 따라 $2^{\times \text{의 개수}}$ 개의 2진수가 생긴다. 그러므로 다시 작성하면 다음과 같다.

$$A \Rightarrow 1000_{(2)}, 1001_{(2)}, 1010_{(2)}, 1011_{(2)}, 1100_{(2)}, 1101_{(2)}, 1110_{(2)}, 1111_{(2)}$$

$$\overline{A}B \Rightarrow 0100_{(2)}, 0101_{(2)}, 0110_{(2)}, 0111_{(2)}$$

이렇게 작성한 2진수들을 다 모으면 함수 F의 1이 나오는 집합 T가 된다.

$$F(A, B, C, D) = \Sigma m(0100_{(2)}, 0101_{(2)}, 0110_{(2)}, 0111_{(2)}, 1000_{(2)}, 1001_{(2)}, 1010_{(2)}, 1011_{(2)}, 1100_{(2)}, 1101_{(2)}, 1110_{(2)}, 1111_{(2)})$$

이를 10진수로 바꾸면 아래와 같이 된다.

$$F(A, B, C, D) = \Sigma m(4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15)$$

또 다른 예시로 $F(A, B, C) = AB + A \odot C$ 라는 논리식에서 결과가 1이 나오는 집합 T를 구해보자.

$F(A, B, C) = AB + A \odot C$ 에서 SOP형식으로 변환

$$AB + \overline{A}\overline{C} + AC \text{에서}$$

$$AB \Rightarrow 11 \times_{(2)} \Rightarrow 110_{(2)}, 111_{(2)}$$

$$\overline{A}\overline{C} \Rightarrow 0 \times 0_{(2)} \Rightarrow 000_{(2)}, 010_{(2)}$$

$$AC \Rightarrow 1 \times 1_{(2)} \Rightarrow 101_{(2)}, 111_{(2)}$$

이 결과에서 겹치는 것을 합쳐서 F에 적으면

$$F(A, B, C) = \Sigma m(000_{(2)}, 010_{(2)}, 101_{(2)}, 110_{(2)}, 111_{(2)})$$

2진수를 10진수로 바꾸면

$$F(A, B, C) = \Sigma m(0, 2, 5, 6, 7)$$

이 방법을 사용하면 주어진 논리식을 $F(A, B, C, \dots, D) = \Sigma m(n_1, n_2, n_3, \dots, n_m)$ 꼴로 바꿀 수 있다.

다만 원함수 F가 $\Sigma m + \Sigma d$ 꼴로 이루어졌으면 Σd 의 집합 D에 있는 요소 중 일부 또는 전체가 들어갔을 가능성이 있다. 이는 논리식을 최대한 단순화 시키기 위하여 무관항을 적극적으로 이용하기 때문에 나타나는 현상이다. 그러므로 원함수가 $F = \Sigma m$ 꼴이 아니라 $F = \Sigma m + \Sigma d$ 꼴이거나 원함수의 형태를 알 수 없는 경우에는 구한 집합 T가 원함수의 Σm 에 들어가는 집합의 크기보다 더 클 수 있다(즉 원함수와 구한 함수의 의도한 결과값 중 일부가 주항이 아니라 무관항일 수도 있다)

그러므로 $F = \Sigma m + \Sigma d$ 꼴이면 해당 논리식을 만든 목적이나 실제 작동하는 입력 변수값 등을 종합적으로 고려하여 Σd 의 집합 D를 유추하여야 한다.

[카르노맵]
 드디어 카르노맵에 대해서 배울 것이다. 카르노맵은 아래의 SOP, POS수식에서 각각 적용되는 2개의 규칙을 이용하여 복잡한 논리식(함수)을 간단하게 정리하는 방법이다.

SOP수식 카르노맵에서 적용되는 규칙
 $fG+f\overline{G}=f$

POS수식 카르노맵에서 적용되는 규칙
 $(f+G)(f+\overline{G})=f$

위의 두 개의 규칙을 보면 카르노맵도 두 가지 방법으로 나뉘는가 싶을 것이다. 하지만 카르노맵은 한번 완성하고 나면 그를 기반으로 SOP수식이나 POS수식으로 자유롭게 바꿀 수 있다.

카르노맵은 함수 F의 결과들을 표에 작성하고 0이나 1의 결과들을 $2^n, n \in \mathbb{N}_0$ 개씩 묶어서 수식으로 변환할 수 있다. 이제부터 자세하게 알아보자.

F(A,B)=Σm(1,3)			
10진수로 변환한 입력 2진수	입력		출력
	A	B	
0	0	0	0
1	0	1	1
2	1	0	0
3	1	1	1

만약 F(A,B)=Σm(1,3)이 있다고 하자. 그렇다면 우리는 이를 진리표로 다음과 같이 표기할 수 있다.

이제부터 이 진리표를 기반으로 카르노맵을 그릴 것이다. 카르노맵은 변수의 개수에 따라 다음과 같이 그려진다.

2개 입력에 따른 카르노맵		
B \ A	0	1
0		
1		

3개 입력에 따른 카르노맵				
C \ AB	00	01	11	10
0				
1				

4개 입력에 따른 카르노맵				
AB \ CD	00	01	11	10
00				
01				
11				
10				

여기에서 $F(A,B)=\Sigma m(1,3)$ 는 입력이 두 개이므로 아래의 2개 입력에 따른 카르노맵을 쓰겠다. 빈 곳에는 F가 1인 경우에는 1을, F가 0인 경우에는 0을 적으면 된다.

F(A,B)= $\Sigma m(1,3)$ 의 카르노맵		
A \ B	0	1
0	0	0
1	1	1

이제 1을 기준으로 2ⁿ개씩 묶어보자

F(A,B)= $\Sigma m(1,3)$ 의 카르노맵		
A \ B	0	1
0	0	0
1	1	1

이제 묶은 곳을 잘 보자. 보라색 부분에서는 A가 0일 때와 1일 때가 같이 존재한다. 이 말은 보라색 부분에서는 A와 B의 입력이 둘 다 중요한 것이 아닌 B의 입력만 중요하다는 의미가 된다. 왜냐하면 보라색 부분에서는 A값에 무관하게 B가 1일 때에만 결과로 1을 내보내기 때문이다. 그러므로 이를 SOP형식으로 적으면 F는 다음과 같다.

$$F(A,B) = B$$

이 결과가 맞는지 한번 이전에 배운 내용으로 계산해보자.

$$F(A,B) = \Sigma(1,3)\text{에서}$$

$$\overline{A}B + AB = B(\overline{A} + A) = B \circledast \text{이므로}$$

$$\therefore F(A,B) = B$$

계산 결과 카르노맵과 같은 결과가 나왔다.

이번에는 0을 기준으로 2ⁿ개씩 묶어보자

F(A,B)=Σm(1,3)의 카르노맵		
A \ B	0	1
0	0	0
1	1	1

보라색으로 묶은 곳을 보면 A의 값과 무관하게 B가 0이 나오는 것을 볼 수 있다. 이를 POS형식으로 적으면 다음과 같다.

$$F(A,B) = B$$

이번에는 F(A,B)=Σm(1,2,3)의 간소화된 논리식을 카르노맵으로 구해보자. F(A,B)=Σm(1,2,3)은 알다시피 OR연산이다. 이를 이해하고 진리표 작성 후 카르노맵을 그려보자.

F(A,B)=Σm(1,2,3)의 카르노맵		
A \ B	0	1
0	0	1
1	1	1

여기에서는 기본 연산이므로 진리표는 생략하였다. 1을 기준으로 2ⁿ개씩 묶으면 각각 다음과 같다.

F(A,B)=Σm(1,2,3)의 카르노맵		
A \ B	0	1
0	0	1
1	1	1

F(A,B)=Σm(1,2,3)의 카르노맵		
A \ B	0	1
0	0	1
1	1	1

보라색 묶음을 1번 묶음, 청록색 묶음을 2번 묶음이라고 하자. 그럼 1번 묶음은 A의 값과 무관하게 B의 값이 정해지므로 SOP형식으로 표현하면 B 이다. 그리고 2번 묶음은 B의 값과 무관하게 A의 값이 정해지므로 SOP형식으로 표현하면 A 이다. 이를 모아서 SOP형식으로 F의 논리식을 적으면 다음과 같다.

$$F(A,B)=A+B$$

이로써 F가 OR연산과 같은 함수였다는 것을 알 수 있다.

이번에는 0을 기준으로 2^n 개씩 묶어보자

F(A,B)=Σm(1,2,3)의 카르노맵		
A \ B	0	1
0	0	1
1	1	1

여기에서 보라색 묶음은 A=0, B=0인 곳에만 존재한다. 이 말은 이 묶음에서 0이 나옴에 있어서 상관없는 변수는 없다는 것이다. 그런 보라색 묶음을 POS형식으로 표현하면 $A+B$ 이다. 다른 묶음이 존재하지 않으므로 이는 F의 논리식과 같다.

이번에는 $F(A,B,C)=\Sigma m(T)+\Sigma d(D)$ 꼴의 함수를 카르노맵으로 그려보자. 이번에는 처음부터 카르노맵에서 논리회로를 구상했다고 가정하고 카르노맵부터 시작하겠다.

F(A,B,C)=Σm(T)+Σd(D)의 카르노맵				
C \ AB	00	01	11	10
0	1	0	×	1
1	1	1	0	1

이 카르노맵을 보면 $110_{(2)}$ 는 결과값이 무관한 무관항임을 알 수 있다. 우리의 목표는 최대한 0이나 1의 집합에 있는 모든 요소를 포함해서 최대한 크고 적게 묶어야 한다(이때 이미 묶인 요소도 다른 묶음에 포함할 수 있다) 이렇게 하면 논리식이 최대한 간결해지기 때문이다.

SOP형식으로 구할 시 $110_{(2)}$ 은 1로 가정하면 새로운 작은 묶음만 만들고 기존에 있던 여러 요소를 묶어서 거대한 묶음을 만들 수 없어서 이득이 크지 않을 것 같으므로 0으로 가정하고 묶어보겠다.

F(A,B,C)=Σm(T)+Σd(D)의 카르노맵				
AB \ C	00	01	11	10
0	1	0	×	1
1	1	1	0	1

F(A,B,C)=Σm(T)+Σd(D)의 카르노맵				
AB \ C	00	01	11	10
0	1	0	×	1
1	1	1	0	1

보라색 묶음을 1번 묶음, 청록색 묶음을 2번 묶음이라고 하자. 참고로 인접한 숫자끼리 2^n 개씩 묶어야 하므로 1번 묶음은 올바른 묶음이다.

1번 묶음은 SOP형식으로 표현하면 $\overline{B}$ 이다. 그리고 2번 묶음은 SOP형식으로 표현하면 $\overline{A}C$ 이다. 그러므로 F는 다음과 같다.

$$F(A,B,C) = \Sigma m(T) + \Sigma d(D) \Rightarrow \overline{A}C + \overline{B}$$

등호가 아닌 조건문으로 표기한 이유는 F의 결과는 Σd에 의해 여러 가지가 나올 수 있고, 그 여러 가지 답이 F=Σm 꼴로 가정했을 때 다른 논리식이므로 조건문으로 표현하였다.

변수가 2개, 3개일 때와 같은 방법으로 변수가 4개일 때도 카로노맵을 통하여 구할 수 있다.

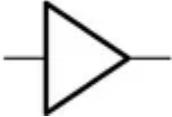
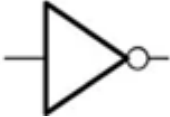
이번에는 카르노맵이 무용지물이 되는 경우에 대해서 알아보자.

F(A,B,C,D)=Σm(T)의 카르노맵				
AB \ CD	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

이 함수 $F(A,B,C,D)$ 는 어떻게 묶어도 최대 하나만 있는 형태 말고는 묶을 방법이 없다. 이 경우에는 간략화되는 항이 하나도 없으므로 진리표나 $\Sigma m(T)$ 에서 집합 T 에 속하는 경우를 전부 빠짐없이 다 적을 수밖에 없다.

[논리회로 그리기]

이번에는 머리를 식히는 겸 논리회로 그리기에 대해서 배우겠다. 논리게이트의 기호는 미국 국립 표준 협회(ANSI)에서 제정한 기호로 표기하겠다.

BUF(버퍼)			NOT Gate		
					
입력		출력	입력		출력
0		0	0		1
1		1	1		0

AND Gate			NAND Gate		
					
입력		출력	입력		출력
A	B	Y	A	B	Y
0	0	0	0	0	1
0	1	0	0	1	1
1	0	0	1	0	1
1	1	1	1	1	0

OR Gate			NOR Gate		
					
입력		출력	입력		출력
A	B	Y	A	B	Y
0	0	0	0	0	1
0	1	1	0	1	0
1	0	1	1	0	0
1	1	1	1	1	0

XOR Gate			XNOR Gate		
					
입력		출력	입력		출력
A	B	Y	A	B	Y
0	0	0	0	0	1
0	1	1	0	1	0
1	0	1	1	0	0
1	1	0	1	1	1

위의 논리게이트 기호들을 외워보자

8bit 메모리는 2^4 개 있음.

$F(using\ memory\ start\ point_{(4bit)}, using\ memory\ endpoint_{(4bit)})$: 메모리 읽기

$m(memory_{(4bit)}, next\ memory\ point_{(4bit)})$: 일반 메모리 구성

$M(m_{start}, m_{command\ start}, m_{command}, ..., m_{end})$: 읽은 메모리 집합

m_{start} = 메모리 집합 크기

$m_{command\ start}$ = 명령어 사용 횟수

$m_{command} + m_{extra}$ = 명령어1 + 명령어1에 대한 부속 명령

$m_{command2} + m_{extra}$ = 명령어2 + 명령어2에 대한 부속 명령

$\vdots$

m_x = 명령어 종료 문구($1101_{(2)}$)

m_{x+1} = 데이터

$\vdots$

m_{end} = 데이터 종료 문구($1011_{(2)}$ 2번 반복)

예를 들어

AND명령 : AND_{0010} (연산할 숫자 갯수, 숫자1 주소, 숫자2 주소...)

시작할 때 0번 메모리부터 F 로 읽고 메모리 구성이 다음과 같다면

0번 : 0001 1111

1번 : 1010 0010

2번 : 0001 0011

3번 : 0010 0100

4번 : 0011 0101

5번 : 1010 0110

6번 : 1011 0111

7번 : 1100 1001

8번 : $null$

9번 : 1101 1010

10번 : 0010 1011

11번 : 0111 1100

12번 : 1010 1111

15번 : 1011 1011

결과 : $0010_{(2)}$

ABCD

1010

1011

1100

1101

1110

1111

101 ×

110 ×

111 ×

101 ×

11 × ×

$A\overline{B}C + AB = A(\overline{B}C + B) = A(\overline{B} + B)(C + B) = A(C + B)$

